6901012610196 Chonpatchut Boonpalit Cpr.E S.2

ข้อ 1 การค้นหา Character จากด้านซ้ายของ String

ข้อนี้ทำอะไร

ข้อนี้เป็นการเขียนฟังก์ชันสำหรับค้นหา Character ที่กำหนดภายใน String โดยเริ่มค้นหาจากด้านซ้าย หรือ `index = 0` เมื่อพบ Character ที่ต้องการให้คืนค่า Index ของตำแหน่งนั้น หากค้นหาจนครบ String แล้วไม่พบ Character ที่ต้องการ ให้คืนค่า `-1` ตามโจทย์ Lab 6

Code

```
def searchLeft(text, char):
    index = 0

    while index < len(text):
        if text[index] == char:
            return index

        index += 1

    return -1
```

อธิบายการทำงานของ Code

บรรทัดที่ 1

```
def searchLeft(text, char):
```

เป็นการสร้างฟังก์ชันชื่อ `searchLeft` โดยรับข้อมูลเข้ามา 2 ตัว ได้แก่ `text` ซึ่งเป็น String ที่ต้องการค้นหา และ `char` ซึ่งเป็น Character ที่ต้องการค้นหา

บรรทัดที่ 2

```
index = 0
```

กำหนดค่าเริ่มต้นของตัวแปร `index` เป็น 0 เนื่องจากต้องการเริ่มค้นหาจากตัวอักษรตัวแรกของ String

บรรทัดที่ 4

```
while index < len(text):
```

ใช้ `while loop` เพื่อวนตรวจสอบตัวอักษรใน String โดย `len(text)` ใช้หาความยาวของ String และกำหนดให้ `index` ต้องน้อยกว่าความยาวของ String เพื่อป้องกันการเข้าถึงตำแหน่งที่ไม่มีอยู่จริง

บรรทัดที่ 5

if text[index] == char:

ตรวจสอบว่า Character ในตำแหน่งปัจจุบันตรงกับ Character ที่ต้องการค้นหาหรือไม่

บรรทัดที่ 6

return index

ถ้าพบ Character ที่ต้องการ จะคืนค่า Index ของตำแหน่งนั้นและจบการทำงานของฟังก์ชันทันที

บรรทัดที่ 8

index += 1

หาก Character ในตำแหน่งปัจจุบันไม่ตรงกับ Character ที่ต้องการ จะเพิ่มค่า `index` ขึ้น 1 เพื่อไปตรวจสอบตัวอักษรถัดไป

บรรทัดที่ 10

return -1

หากวนตรวจสอบ String จนครบแล้วไม่พบ Character ที่ต้องการ จะคืนค่า `-1` เพื่อแสดงว่าไม่พบ Character ดังกล่าว

ตัวอย่างการทำงาน

print(searchLeft("Thailand", "a"))

ผลลัพธ์

2

เนื่องจาก String `"Thailand"` มี `a` ตัวแรกอยู่ที่ Index 2

Error ที่พบ

ปัญหาที่อาจพบคือ `IndexError` ซึ่งเกิดจากการกำหนดเงื่อนไขของ `while` ผิด เช่น

while index <= len(text):

สาเหตุคือ Index สุดท้ายของ String จะน้อยกว่า `len(text)` อยู่ 1 ดังนั้นควรใช้

while index < len(text):

อีกปัญหาหนึ่งคือการลืมเขียน

index += 1

ซึ่งจะทำให้ค่า `index` ไม่เปลี่ยนแปลงและเกิดการวน Loop ไม่สิ้นสุด

ข้อ 2 การค้นหา Character จากด้านขวาของ String

ข้อนี้ทำอะไร

ข้อนี้เป็นการเขียนฟังก์ชันสำหรับค้นหา Character ใน String โดยเริ่มค้นหาจากด้านขวา หรือจากตัวอักษรตัวสุดท้ายของ String เมื่อพบ Character ที่ต้องการให้คืนค่า Index ของตำแหน่งนั้น หากไม่พบให้คืนค่า `-1`

Code

```
def searchRight(text, char):
    index = len(text) - 1

    while index >= 0:
        if text[index] == char:
            return index

        index -= 1

    return -1
```

อธิบายการทำงานของ Code

บรรทัดที่ 1

def searchRight(text, char):

สร้างฟังก์ชันชื่อ `searchRight` โดยรับ String และ Character ที่ต้องการค้นหา

บรรทัดที่ 2

index = len(text) - 1

กำหนดให้ `index` เริ่มต้นที่ตำแหน่งสุดท้ายของ String เนื่องจากต้องการค้นหาจากด้านขวา

ตัวอย่างเช่น ถ้า String มีความยาว 8 ตัวอักษร Index สุดท้ายจะเป็น 7

บรรทัดที่ 4

while index >= 0:

ใช้ `while loop` เพื่อวนค้นหาจากด้านขวาไปด้านซ้าย โดยจะทำงานตราบใดที่ `index` ยังมากกว่าหรือเท่ากับ 0

บรรทัดที่ 5

if text[index] == char:

เปรียบเทียบ Character ในตำแหน่งปัจจุบันกับ Character ที่ต้องการค้นหา

บรรทัดที่ 6

return index

หากพบ Character จะคืนค่า Index ทันที

บรรทัดที่ 8

index -= 1

หากยังไม่พบ Character จะลดค่า Index ลง 1 เพื่อเลื่อนไปตรวจสอบตัวอักษรทางด้านซ้าย

บรรทัดที่ 10

return -1

หากค้นหาจนครบ String แล้วไม่พบ Character จะคืนค่า `-1`

ตัวอย่างการทำงาน

print(searchRight("Thailand", "a"))

ผลลัพธ์

5

เนื่องจาก `"Thailand"` มีตัว `a` อยู่ที่ Index 2 และ Index 5 แต่ฟังก์ชันนี้เริ่มค้นหาจากด้านขวา จึงพบตัวที่ Index 5 ก่อน

Error ที่พบ

ข้อผิดพลาดที่สำคัญคือการใช้

index = len(text)

แทน

index = len(text) - 1

เนื่องจาก `len(text)` คือจำนวนตัวอักษร ไม่ใช่ Index ตัวสุดท้าย

ตัวอย่าง String ที่มีความยาว 5 ตัวอักษร จะมี Index ตั้งแต่ 0 ถึง 4 ดังนั้น Index สุดท้ายคือ `5 - 1 = 4`

ข้อ 3 การตรวจสอบ Palindrome

ข้อนี้ทำอะไร

ข้อนี้เป็นการเขียน Boolean Function เพื่อตรวจสอบว่า String เป็น Palindrome หรือไม่ โดยใช้ `while loop`

Palindrome คือ String ที่เมื่ออ่านจากด้านหน้าไปด้านหลังและจากด้านหลังไปด้านหน้าแล้วมีลำดับเหมือนกัน

ตัวอย่างเช่น

level

เมื่ออ่านจากซ้ายไปขวาและขวาไปซ้ายจะได้คำว่า `level` เหมือนกัน จึงเป็น Palindrome

Code

```
def isPalindrome(text):
    text = text.lower()
```

```
    left = 0
    right = len(text) - 1

    while left < right:
        if text[left] != text[right]:
            return False

        left += 1
        right -= 1

    return True
```

อธิบายการทำงานของ Code

บรรทัดที่ 1

def isPalindrome(text):

สร้างฟังก์ชันชื่อ `isPalindrome` เพื่อใช้ตรวจสอบว่า String เป็น Palindrome หรือไม่

บรรทัดที่ 2

text = text.lower()

เปลี่ยนตัวอักษรทั้งหมดใน String ให้เป็นตัวพิมพ์เล็ก เพื่อไม่ให้ตัวพิมพ์ใหญ่และตัวพิมพ์เล็กทำให้การเปรียบเทียบแตกต่างกัน เช่น `Deed` จะถูกเปลี่ยนเป็น `deed`

บรรทัดที่ 4

left = 0

กำหนดตัวชี้ด้านซ้ายให้เริ่มจาก Index 0

บรรทัดที่ 5

right = len(text) - 1

กำหนดตัวชี้ด้านขวาให้เริ่มจาก Index สุดท้าย

บรรทัดที่ 7

while left < right:

วนตรวจสอบตัวอักษรจากด้านซ้ายและด้านขวาเข้าหากัน โดยไม่จำเป็นต้องตรวจสอบต่อเมื่อ `left` และ `right` มาถึงตำแหน่งเดียวกันหรือข้ามกัน

บรรทัดที่ 8

if text[left] != text[right]:

เปรียบเทียบตัวอักษรด้านซ้ายกับตัวอักษรด้านขวา หากไม่เหมือนกัน แสดงว่า String ไม่ใช่ Palindrome

บรรทัดที่ 9

return False

คืนค่า `False` เพื่อบอกว่า String ไม่ใช่ Palindrome

บรรทัดที่ 11

left += 1

เลื่อนตัวชี้ด้านซ้ายไปทางขวา 1 ตำแหน่ง

บรรทัดที่ 12

right -= 1

เลื่อนตัวชี้ด้านขวาไปทางซ้าย 1 ตำแหน่ง

บรรทัดที่ 14

return True

หากตรวจสอบทุกคู่แล้วไม่มีตัวอักษรคู่ใดแตกต่างกัน จะคืนค่า `True` แสดงว่า String เป็น Palindrome

ตัวอย่างการทำงาน

print(isPalindrome("Deed"))

ผลลัพธ์

True

เนื่องจากโปรแกรมเปลี่ยน `"Deed"` เป็น `"deed"` ก่อน แล้วจึงเปรียบเทียบ

d e e d
↑ ↑
d = d

e e
↑ ↑
e = e

จึงเป็น Palindrome

Error ที่พบ

ข้อผิดพลาดที่อาจพบคือการลืมเพิ่มหรือลดค่า `left` และ `right`

left += 1
right -= 1

หากไม่เขียนสองบรรทัดนี้ โปรแกรมอาจเกิด Infinite Loop เนื่องจากค่าที่ใช้ในเงื่อนไข `while` ไม่เปลี่ยนแปลง

อีกปัญหาคือการไม่จัดการตัวพิมพ์ใหญ่และตัวพิมพ์เล็ก ทำให้ `"Deed"` และ `"deed"` ถูกมองว่าแตกต่างกัน จึงใช้ `lower()` เพื่อให้สามารถเปรียบเทียบได้โดยไม่สนใจตัวพิมพ์

ข้อ 4 ฟังก์ชัน countChar

ข้อนี้ทำอะไร

ข้อนี้ให้สร้างฟังก์ชันชื่อ `countChar` เพื่อใช้สำหรับนับจำนวน Character ที่กำหนดว่าปรากฏอยู่ใน String กี่ครั้ง

ตัวอย่าง

countChar("a", "Thailand")

ผลลัพธ์คือ `2` เนื่องจากคำว่า `Thailand` มีตัวอักษร `a` จำนวน 2 ตัว

Code

```
def countChar(char, text):
    count = 0
    index = 0
```

```
    while index < len(text):
        if text[index] == char:
            count += 1

        index += 1

    return count
```

อธิบายการทำงานของ Code

บรรทัดที่ 1

```
def countChar(char, text):
```

สร้างฟังก์ชันชื่อ `countChar` โดยรับข้อมูล 2 ตัว คือ Character ที่ต้องการนับ และ String ที่ต้องการตรวจสอบ

บรรทัดที่ 2

```
count = 0
```

สร้างตัวแปร `count` เพื่อเก็บจำนวน Character ที่พบ และกำหนดค่าเริ่มต้นเป็น 0

บรรทัดที่ 3

```
index = 0
```

กำหนด Index เริ่มต้นเป็น 0 เพื่อเริ่มตรวจสอบ String จากตัวแรก

บรรทัดที่ 5

```
while index < len(text):
```

วนตรวจสอบ Character ทุกตัวใน String

บรรทัดที่ 6

```
if text[index] == char:
```

ตรวจสอบว่า Character ในตำแหน่งปัจจุบันตรงกับ Character ที่ต้องการนับหรือไม่

บรรทัดที่ 7

```
count += 1
```

ถ้าพบ Character ที่ต้องการ ให้เพิ่มจำนวน `count` ขึ้น 1

บรรทัดที่ 9

index += 1

เลื่อนไปตรวจสอบ Character ตัวถัดไป

บรรทัดที่ 11

return count

เมื่อวนตรวจสอบครบทุก Character แล้ว ให้คืนค่าจำนวนที่นับได้

ตัวอย่างการทำงาน

print(countChar("a", "Thailand"))

ผลลัพธ์

2

Error ที่พบ

หากลืมเขียน

count += 1

โปรแกรมจะสามารถวนตรวจสอบ String ได้ แต่ค่าที่คืนออกมาจะเป็น 0 เนื่องจากไม่มีคำสั่งเพิ่มค่าตัวนับ

อีกปัญหาคือการส่ง Argument ผิดลำดับ เช่น

countChar("Thailand", "a")

เนื่องจากฟังก์ชันกำหนดให้ Argument ตัวแรกเป็น `char` และตัวที่สองเป็น `text` ดังนั้นการเรียกที่ถูกต้องคือ

countChar("a", "Thailand")

ข้อ 5 ฟังก์ชัน countVowels

ข้อนี้ทำอะไร

ข้อนี้ให้สร้างฟังก์ชันชื่อ `countVowels` เพื่อใช้สำหรับนับจำนวนสระภาษาอังกฤษ ได้แก่ `a`, `e`, `i`, `o` และ `u` ภายใน String โดยโจทย์กำหนดว่าฟังก์ชัน `countVowels()` ต้องมีการเรียกใช้ `countChar()` ด้วย

ตัวอย่าง

countVowels("Bangkok")

จะได้ผลลัพธ์ `2` เพราะมีสระ `a` และ `o`

Code

```
def countVowels(text):
    vowels = "aeiou"
    count = 0
    index = 0

    while index < len(vowels):
        count += countChar(vowels[index], text)
        index += 1

    return count
```

อธิบายการทำงานของ Code

บรรทัดที่ 1

def countVowels(text):

สร้างฟังก์ชันชื่อ `countVowels` โดยรับ String ที่ต้องการตรวจสอบเข้ามา

บรรทัดที่ 2

vowels = "aeiou"

สร้าง String ที่เก็บสระภาษาอังกฤษทั้ง 5 ตัวที่ต้องการตรวจสอบ

บรรทัดที่ 3

count = 0

สร้างตัวแปรสำหรับเก็บจำนวนสระทั้งหมดที่พบ

บรรทัดที่ 4

index = 0

กำหนด Index เริ่มต้นสำหรับวนผ่าน String `vowels`

บรรทัดที่ 6

while index < len(vowels):

วนตรวจสอบสระทีละตัว ตั้งแต่ `a` ไปจนถึง `u`

บรรทัดที่ 7

count += countChar(vowels[index], text)

เป็นคำสั่งสำคัญของฟังก์ชัน โดยเรียกใช้ `countChar()` เพื่อหาจำนวนของสระแต่ละตัวใน String แล้วนำผลลัพธ์ไปบวกกับ `count`

สำหรับ `"Bangkok"` จะมีการทำงานดังนี้

countChar("a", "Bangkok") → 1
countChar("e", "Bangkok") → 0
countChar("i", "Bangkok") → 0
countChar("o", "Bangkok") → 1
countChar("u", "Bangkok") → 0

ดังนั้นจำนวนสระทั้งหมดคือ

1 + 0 + 0 + 1 + 0 = 2

บรรทัดที่ 8

index += 1

เพิ่ม Index เพื่อไปตรวจสอบสระตัวถัดไป

บรรทัดที่ 10

return count

คืนค่าจำนวนสระทั้งหมดที่พบ

ตัวอย่างการทำงาน

print(countVowels("Bangkok"))

ผลลัพธ์

2

Error ที่พบ

ข้อผิดพลาดที่สำคัญคือการเขียนฟังก์ชันโดยไม่เรียก `countChar()` เช่นการตรวจสอบสระโดยตรง แม้ว่าจะสามารถหาคำตอบได้ แต่ไม่ตรงตามเงื่อนไขของโจทย์ที่กำหนดให้ `countVowels()` ต้องเรียกใช้ `countChar()`

อีกข้อผิดพลาดคือการใช้ `len(text)` ใน Loop แทน `len(vowels)` เพราะ Loop นี้มีหน้าที่วนผ่านสระ `aeiou` ไม่ใช่วนผ่าน String ที่นำมาตรวจสอบ

ข้อ 6 ฟังก์ชัน countWords

ข้อนี้ทำอะไร

ข้อนี้ให้สร้างฟังก์ชันชื่อ `countWords` เพื่อใช้สำหรับนับจำนวนคำที่อยู่ใน String

ตัวอย่างจากโจทย์คือ

Human and AI can make mistakes

สามารถแบ่งออกเป็น 6 คำ ได้แก่

1. Human
2. and
3. AI
4. can
5. make
6. mistakes

ดังนั้นผลลัพธ์คือ `6`

Code

```
def countWords(text):
    count = 0
    index = 0
    in_word = False

    while index < len(text):
        if text[index] != " " and in_word == False:
            count += 1
            in_word = True

        elif text[index] == " ":
            in_word = False

        index += 1

    return count
```

อธิบายการทำงานของ Code

บรรทัดที่ 1

```
def countWords(text):
```

สร้างฟังก์ชันชื่อ `countWords` โดยรับ String ที่ต้องการนับจำนวนคำ

บรรทัดที่ 2

```
count = 0
```

สร้างตัวแปรสำหรับเก็บจำนวนคำ โดยเริ่มต้นเป็น 0

บรรทัดที่ 3

```
index = 0
```

กำหนดให้เริ่มตรวจสอบ String จาก Index 0

บรรทัดที่ 4

```
in_word = False
```

สร้างตัวแปรสำหรับบอกสถานะว่าในขณะนั้นกำลังอยู่ภายในคำหรือไม่ โดยเริ่มต้นเป็น `False`

บรรทัดที่ 6

while index < len(text):

ใช้ `while loop` เพื่อวนตรวจสอบ Character ทุกตัวใน String

บรรทัดที่ 7

if text[index] != " " and in_word == False:

ตรวจสอบว่าตัวอักษรปัจจุบันไม่ใช่ช่องว่าง และก่อนหน้านี้ยังไม่ได้อยู่ภายในคำ

ถ้าเป็นจริง แสดงว่า Character ตัวนี้เป็นจุดเริ่มต้นของคำใหม่

บรรทัดที่ 8

count += 1

เพิ่มจำนวนคำขึ้น 1

บรรทัดที่ 9

in_word = True

กำหนดสถานะให้เป็น `True` เพื่อบอกว่าขณะนี้กำลังอยู่ภายในคำแล้ว

บรรทัดที่ 11

elif text[index] == " ":

ตรวจสอบว่าตัวอักษรปัจจุบันเป็นช่องว่างหรือไม่

บรรทัดที่ 12

in_word = False

เมื่อพบช่องว่าง แสดงว่าคำนั้นจบแล้ว จึงเปลี่ยนสถานะกลับเป็น `False`

เมื่อเจอตัวอักษรตัวถัดไป โปรแกรมจะสามารถนับเป็นคำใหม่ได้

บรรทัดที่ 14

index += 1

เพิ่ม Index เพื่อไปตรวจสอบ Character ถัดไป

บรรทัดที่ 16

return count

เมื่อวนตรวจสอบครบทั้ง String แล้ว ให้คืนค่าจำนวนคำทั้งหมด

ตัวอย่างการทำงาน

print(countWords("Human and AI can make mistakes"))

ผลลัพธ์

6

Error ที่พบ

ปัญหาที่อาจเกิดขึ้นคือการนับจำนวนช่องว่างแทนจำนวนคำ เช่นใช้

```
if text[index] == " ":
    count += 1
```

วิธีนี้ไม่เหมาะสม เพราะจำนวนช่องว่างไม่ได้เท่ากับจำนวนคำเสมอไป โดยเฉพาะเมื่อมีช่องว่างหลายตัวติดกัน

ตัวอย่าง

Hello World

มี 2 คำ แต่มีช่องว่าง 4 ตัว

ดังนั้นจึงใช้ตัวแปร `in_word` เพื่อช่วยตรวจสอบจุดเริ่มต้นของแต่ละคำ

ข้อ 7 โจทย์เพิ่มเติมที่คิดว่าน่าสนใจ

การหาตัวอักษรที่ปรากฏมากที่สุดใน String

ข้อนี้ทำอะไร

โจทย์เพิ่มเติมที่คิดว่าน่าสนใจและควรเพิ่มใน Lab นี้ คือการสร้างฟังก์ชัน `mostFrequentChar()` เพื่อหาว่า Character ใดปรากฏใน String มากที่สุด และปรากฏทั้งหมดกี่ครั้ง

ตัวอย่างเช่น

Input:
banana

ในคำว่า banana

b = 1 ครั้ง
a = 3 ครั้ง
n = 2 ครั้ง

ดังนั้นผลลัพธ์คือ

Character: a
Count: 3

Code

```
def mostFrequentChar(text):
    maxChar = ""
    maxCount = 0
    index = 0

    while index < len(text):
        char = text[index]
        count = countChar(char, text)

        if count > maxCount:
            maxCount = count
            maxChar = char

        index += 1

    return maxChar, maxCount
```

อธิบายการทำงานของ Code

บรรทัดที่ 1

```
def mostFrequentChar(text):
```

สร้างฟังก์ชันชื่อ `mostFrequentChar` เพื่อหาตัวอักษรที่ปรากฏมากที่สุดใน String

บรรทัดที่ 2

maxChar = ""

สร้างตัวแปรสำหรับเก็บ Character ที่มีจำนวนมากที่สุด โดยเริ่มต้นเป็น String ว่าง

บรรทัดที่ 3

maxCount = 0

สร้างตัวแปรสำหรับเก็บจำนวนครั้งสูงสุดที่พบ โดยเริ่มต้นเป็น 0

บรรทัดที่ 4

index = 0

กำหนด Index เริ่มต้นเป็น 0

บรรทัดที่ 6

while index < len(text):

วนตรวจสอบ Character ทุกตัวใน String

บรรทัดที่ 7

char = text[index]

นำ Character ณ ตำแหน่งปัจจุบันมาเก็บไว้ในตัวแปร `char`

บรรทัดที่ 8

count = countChar(char, text)

เรียกใช้ฟังก์ชัน `countChar()` เพื่อหาว่า Character ตัวปัจจุบันปรากฏอยู่ใน String กี่ครั้ง

บรรทัดที่ 10

if count > maxCount:

เปรียบเทียบจำนวนที่เพิ่งนับได้กับจำนวนสูงสุดที่เคยพบ

บรรทัดที่ 11

maxCount = count

ถ้าจำนวนที่เพิ่งนับได้มากกว่า `maxCount` ให้เปลี่ยนค่า `maxCount` เป็นจำนวนใหม่

บรรทัดที่ 12

maxChar = char

เก็บ Character ที่มีจำนวนมากที่สุดไว้ใน `maxChar`

บรรทัดที่ 14

index += 1

เพิ่ม Index เพื่อไปตรวจสอบ Character ตัวถัดไป

บรรทัดที่ 16

return maxChar, maxCount

คืนค่า Character ที่ปรากฏมากที่สุดและจำนวนครั้งที่ Character นั้นปรากฏ

ตัวอย่างการทำงาน

```
char, count = mostFrequentChar("banana")

print("Character:", char)
print("Count:", count)
```

ผลลัพธ์

```
Character: a
Count: 3
```

Error ที่พบ

ข้อผิดพลาดที่อาจเกิดขึ้นคือการไม่นำ `countChar()` มาใช้ ทำให้ต้องเขียน Algorithm สำหรับนับ Character ซ้ำอีกครั้ง ทั้งที่มีฟังก์ชัน `countChar()` อยู่แล้ว

การเรียกใช้ `countChar()` ทำให้สามารถนำ Function เดิมมาประยุกต์ใช้กับโจทย์ใหม่ได้ และแสดงให้เห็นถึงการแบ่งงานของแต่ละ Function อย่างชัดเจน

อีกกรณีหนึ่งคือถ้ามี Character หลายตัวที่มีจำนวนเท่ากัน โปรแกรมนี้จะเลือก Character ตัวแรกที่พบ เพราะใช้เงื่อนไข

if count > maxCount:

ดังนั้นหากต้องการให้แสดง Character ทุกตัวที่มีจำนวนสูงสุด จะต้องปรับ Algorithm เพิ่มเติม

สรุปสิ่งที่ได้จากแต่ละข้อ

ข้อ 1 `searchLeft()`

ฝึกการใช้ String และ Index โดยเริ่มค้นหาจากด้านซ้าย และใช้ `while loop` ในการตรวจสอบ Character ทีละตำแหน่ง

ข้อ 2 `searchRight()`

ฝึกการค้นหา String จากด้านขวา โดยใช้ Index เริ่มต้นเป็น `len(text) - 1` และลด Index ลงทีละ 1

ข้อ 3 `isPalindrome()`

ฝึกการออกแบบ Algorithm โดยใช้ตัวชี้สองตัว คือ `left` และ `right` เพื่อเปรียบเทียบ Character จากด้านหน้าและด้านหลังเข้าหากัน

ข้อ 4 `countChar()`

ฝึกการวน String และการสร้างตัวนับเพื่อหาจำนวนครั้งที่ Character ที่กำหนดปรากฏใน String

ข้อ 5 `countVowels()`

ฝึกการเรียกใช้ Function ภายใน Function โดยนำ `countChar()` มาประยุกต์ใช้เพื่อหาจำนวนสระ `aeiou`

ข้อ 6 `countWords()`

ฝึกการสร้าง Algorithm สำหรับนับจำนวนคำ โดยใช้ `while loop` และตัวแปร `in_word` เพื่อแยกจุดเริ่มต้นและจุดสิ้นสุดของแต่ละคำ

ข้อ 7 `mostFrequentChar()`

เป็นโจทย์เพิ่มเติมที่นำความรู้จากหลายข้อมาใช้ร่วมกัน ได้แก่ String, Index, Loop, Function และ `countChar()` ทำให้ได้ฝึกการคิด Algorithm ที่ซับซ้อนขึ้น